

INTERNATIONAL POSTCODE SYSTEM (IPCS)

von

Bartosz Cieřlicki

Rochdale 2018

Systemannahmen

Das Hauptkonzept des Systems besteht darin, ein einheitliches System von Postleitzahlen für die gesamte Erde zu schaffen. Das wichtigste Ziel ist:

KODIERUNG DER GEOGRAFISCHEN KOORDINATEN JEDER POSTLEITZAHL IN IHREM EIGENEN DATENSATZ.

Das so aufgebaute System macht eine große globale Datenbank mit dem Standort jeder Postleitzahl überflüssig. Es reduziert oder eliminiert zudem die Kosten der Geokodierung und beschleunigt Computersysteme erheblich, die nach einer geografischen Position für eine angefragte Adresse suchen.

Annahmen:

- Die durch die Postleitzahl identifizierte Fläche sollte 1000 m² nicht überschreiten, damit der gewünschte Ort mithilfe einer Karte oder Satellitennavigation leicht und genau erreicht werden kann.
- Das System muss jeden Teil des Planeten abdecken, unabhängig von Bevölkerung, Bebauungsdichte, Gelände oder Oberflächenart.
- Postleitzahlen müssen anhand eines Algorithmus berechnet bzw. erzeugt werden, nicht willkürlich vergeben.
- Aus jedem Postleitzahl-Datensatz lassen sich die geografischen Koordinaten berechnen, die das Zentrum der Fläche angeben.

Das International Postcode System (IPCS):

- sollte auf dem lateinischen Alphabet basieren, da dieses weltweit am verbreitetsten ist.
- sollte eine relativ kleine Anzahl von Zeichen haben, um ein leichtes Einprägen zu ermöglichen.
- sollte keine Sonderzeichen enthalten, die für ein bestimmtes Land spezifisch sind.
- sollte die häufigsten Schreib- und Lesefehler ausschließen (z. B. den Buchstaben O mit der Ziffer 0 und den Buchstaben I mit der Ziffer 1).

Konstruktion der Sektoren

Zur Berechnung des International Postcode System (IPCS) verwenden wir geografische Breite und Länge im Dezimalsystem. Die Länge liegt im Bereich $(-180, 180>$, die Breite im Bereich $(-90, 90>$. Der erste, sehr wichtige Schritt zur Berechnung der internationalen Postleitzahl besteht darin, Breite und Länge auf ein Viertel von 0,001 zu runden.

Beispiel:

Nehmen wir an, die Zahl $t \in (0.000, 0.001>$, die Zahl x ist unsere Rundung. Wenn:

$$x \in (0.000, 0.000125) \rightarrow x = 0.000$$

$$x \in (0.000125, 0.000375) \rightarrow x = 0.00025$$

$$x \in (0.000375, 0.000625) \rightarrow x = 0.0005$$

$$x \in (0.000625, 0.000875) \rightarrow x = 0.00075$$

$$x \in (0.000875, 0.001> \rightarrow x = 0.001$$

Dank der Rundung wird die gesamte Erde in Zonen unterteilt. Die erhaltenen Koordinaten identifizieren jeden erzeugten Sektor auf der Erde und sind zugleich sein Zentrum. Damit ist die eingangs gemachte Annahme erfüllt.

Der Gitterausschnitt sieht folgendermaßen aus:

Die auf Koordinaten basierende Struktur der Sektoren bewirkt, dass die Fläche jedes durch Rundung von Breite und Länge gebildeten Sektors zu den Polen der Erde hin abnimmt. Die größten Flächen liegen daher am Äquator und betragen rund 728 m², was unserer Annahme entspricht.

Wir können die Anzahl der so entstehenden Sektoren berechnen. Die Zahlen werden auf ein Viertel eines Tausendstels gerundet, sodass jedes Tausendstel in 4 Teile geteilt wird.

Für die Länge haben wir den Zahlenbereich $(-180, 180>$. Mit x bezeichnen wir die Anzahl der Unterteilungen. Sie beträgt:

$$x = 360 * 1000 * 4 \quad (1)$$

Für die Breite haben wir den Zahlenbereich $(-90, 90>$. Mit y bezeichnen wir die Anzahl der Unterteilungen. Sie beträgt:

$$y = 181 * 1000 * 4 \quad (2)$$

(Der Wert 181 statt 180 trägt dem Umstand Rechnung, dass der Breitenbereich $(-90, 90>$ beidseitig geschlossen ist — der Südpol (-90°) und der Nordpol (90°) sind zwei verschiedene Punkte — im Gegensatz zum Längenbereich $(-180, 180>$, der nur einseitig geschlossen ist, da -180° und 180° denselben Meridian bezeichnen.)

Mithilfe von (1) und (2) berechnen wir die Anzahl der Sektoren für die gesamte Erde:

$$\text{Number of sectors} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

Es bleibt nur noch, für jeden der gebildeten Sektoren die Internationale Postleitzahl (IPCS) zu konstruieren.

Konstruktion des International Postcode System (IPCS)

Mit lateinischen Buchstaben und Ziffern haben wir 10 Ziffern und 26 Buchstaben. Wir entfernen die Buchstaben O und I aus dem Alphabet (um die Annahmen des Systems zu erfüllen). So erhalten wir folgende 34 Zeichen: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z}. Auf Basis dieser 34 Zeichen erstellen wir ein Zahlensystem – ein 34er-System. Links stehen die Zahlen im 34er-System, rechts das Gegenstück im Dezimalsystem.

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

Jede Zahl im 34er-System lässt sich im Dezimalsystem wie folgt darstellen:

$$\dots dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + \dots$$

, wobei f eine Funktion ist, die jeder Ziffer des 34er-Systems ihr Gegenstück im Dezimalsystem zuordnet (siehe (4)). Dabei sind a, b, c, d Ziffern aus dem 34er-System.

Beispiel:

$$\begin{aligned}
 AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 \\
 &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 \\
 &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 \\
 &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007
 \end{aligned}$$

Wie man sieht, ist die 34er-Notation deutlich kürzer. Wir benötigen 8 Zeichen, um Postleitzahlen für den gesamten Planeten zu erzeugen. Die maximale Zeichenzahl im aktuellen englischen und irischen Postleitzahlensystem beträgt 7, in Japan 7 Ziffern, in den Niederlanden 6 Zeichen usw. Unter Berücksichtigung der Länge von Postleitzahlen in anderen Ländern sowie der Tatsache, dass das International Postcode System ein globales System ist, nehmen wir an, dass 8 Zeichen akzeptabel sind und die Annahmen unseres Systems erfüllen.

Die größte Zahl, die wir mit 8 Ziffern in unserem 34er-System speichern können, ist (detaillierte Berechnungen wurden ausgelassen):

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

Daraus folgt, dass die Erde 1 785 793 904 896 Postleitzahlen haben kann (diese Zahl ist um 1 größer als die größte Zahl, da 00000000 ebenfalls ein Code sein kann). Aus der vorherigen Berechnung (3) kennen wir die Anzahl der Sektoren in unserer Struktur, sie beträgt 1 042 560 000 000. Diese Zahl ist kleiner als 1 785 793 904 895, daher reichen 8 Ziffern im 34er-System aus, um jeden Sektor eindeutig zu identifizieren. Die Sektorflächen sind aufgrund dieser Struktur nicht gleich groß. Die größten liegen nahe dem Äquator und betragen $\approx 728 \text{ m}^2$. Die Fläche der Erde beträgt etwa $510\,100\,000 \text{ km}^2 =$

510 100 000 000 000 m². Um die durchschnittliche Fläche eines Sektors zu berechnen, führen wir eine einfache Operation durch:

$$510\,100\,000\,000\,000\text{ m}^2 / 1\,042\,560\,000\,000 \approx 489.2764\text{ m}^2$$

Somit erfüllt ZZZZZZZZ₃₄ unsere Annahmen, und die Fläche jedes Sektors ist kleiner als 1000 m².

Was nun noch bleibt, ist die Umrechnung des Sektorzentrums in das International Postcode System (IPCS).

Für unsere Zwecke stellen wir den ganzzahligen Anteil negativer Zahlen wie folgt dar:

- Für die Länge wird der negative ganzzahlige Anteil x zu $360 + x$ umgerechnet.
- Für die Breite wird die negative Ganzzahl y zu $180 + y$ umgerechnet.

Der Nachkommaanteil bleibt unverändert. Zusammengefasst:

$$\text{Longitude } -XXX.xxxxx \rightarrow (360 + (-XXX)).xxxxx$$

$$\text{Latitude } -YYY.yyyyy \rightarrow (180 + (-YYY)).yyyyy$$

Beispiele der Umrechnung:

- Die Zahl -75.43725 für die Länge wird zu 285.43725, da $360 + (-75) = 285$.
- Die Zahl -0.411250 für die Länge wird zu 360.411250, da $360 + 0 = 360$.
- Die Zahl -15.44375 für die Breite wird zu 165.44375, da $180 + (-15) = 165$.

Bei der Erstellung des Algorithmus sollten wir berücksichtigen, dass in diesem Datensatz der Südpol bei 90 Grad Breite und der Nordpol bei -90 Grad Breite gleichgesetzt werden.

Unter Berücksichtigung der Umrechnung negativer Zahlen und der Rundung lautet die allgemeine Form der im Dezimalsystem erfassten Koordinaten:

Longitude: XXX.xxxqq, where $XXX \in \{x \in \mathbb{N}: 000 \leq x \leq 360\}$, $xxx \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}$, $qq \in \{00, 25, 50, 75\}$

Latitude: YYY.yyygg, where $YYY \in \{x \in \mathbb{N}: 000 \leq x \leq 180\}$, $yyy \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}$, $gg \in \{00, 25, 50, 75\}$

Im nächsten Schritt wird jedes Element der Zahlen XXX.xxxqq und YYY.yyygg in ein Binärsystem umgewandelt.

$$XXX_{10} = XXXXXXXXX_2 \text{ — max. 360, requires 9 bits.}$$

$$xxx_{10} = xxxxxxxxx_2 \text{ — max. 999, requires 10 bits.}$$

$$qq_{10} = qq_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

$$YYY_{10} = YYYYYYY_2 \text{ — max. 180, requires 8 bits.}$$

$$yyy_{10} = yyyyyyyyy_2 \text{ — max. 999, requires 10 bits.}$$

$$gg_{10} = gg_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

Anschließend ordnen wir die Binärzahlen in folgender Reihenfolge an:

$$XXXXXXXXXX YYYYYYYY xxxxxxxxxx yyyyyyyyyy qq gg_2$$

Wir nehmen eine mindestens 41-Bit-Zahl, setzen sie auf null und tragen die obigen Bits von rechts nach links unter Beibehaltung ihrer Reihenfolge ein. Die erhaltene Zahl ist gleich:

$$XXXXXXXXXXYYYYYYYxxxxxxxxxyyyyyyyyyqqgg_2$$

Im letzten Schritt wandeln wir die Binärzahl in das 34er-System um. Die erhaltene 34er-Zahl ist zugleich unsere Internationale Postleitzahl (IPCS). Die Koordinaten des Zentrums der Fläche sind in unserem International-Postcode-(IPCS)-Datensatz kodiert.

Die Reihenfolge, in der die Bits geschrieben werden, ist sehr wichtig, denn gerundet und gemäß

unserer Konvention dargestellt, ist die Koordinate mit dem größten Zahlenwert 360.99975, 180.99975. Umgewandelt in ein Binärsystem erhalten wir die Zahl:

$$10110100010110100111110011111110011111112 = 1\,549\,224\,509\,055_{10}$$

Die erhaltene Zahl ist kleiner als die maximale Zahl des 34er-Codes, welche $ZZZZZZZZ_{34} = 1\,785\,793\,904\,895_{10}$ beträgt. Eine Änderung der Bit-Reihenfolge könnte diesen Maximalwert überschreiten, was es unmöglich machen würde, Postleitzahlen für alle Sektoren zu erzeugen.

Um die Koordinaten des Sektorzentrums (Internationale Postleitzahl) zu erhalten, führen wir die Operation in umgekehrter Reihenfolge durch.

Beispiele:

Die Internationale Postleitzahl (IPCS) für die Koordinaten -1.347120, 53.983489 konstruieren wir wie folgt:

Wir runden, rechnen die Koordinaten um und erhalten 359.34700, 53.98350

Wir wandeln jedes Element der Koordinaten — also die Zahlen 359, 347, 00, 53, 983, 50 — wie oben beschrieben in ein Binärsystem um und erhalten: 101100111_2 , 0101011011_2 , 00_2 , 01101010_2 , 1111010111_2 , 10_2

Wir ordnen die Zahlen in der richtigen Reihenfolge an: 101100111_2 , 0011010_{21} , 0101011011_2 , 1111010111_2 , 00_2 , 10_2

Wir bilden die Zahl $10110011100110101010101101111110101110010_2$

Wir wandeln die Binärzahl in das 34er-System um und erhalten $VCPM6TKY_{34}$

Die erhaltene Zahl ist die Internationale Postleitzahl (IPCS): VCPM 6TKY

Die Koordinaten für die Internationale Postleitzahl 1GL8 YT7F werden wie folgt berechnet:

Wir wandeln die 34er-Zahl $1GL8YT7F_{34}$ in Binär um und erhalten $000010010001100101100000011010011011010012$

Wir extrahieren die Zahlen 00001001_2 , 0011001_{20} , 1100000011_2 , 0100110110_2 , 10_2 , 01_2

Wir ändern die Reihenfolge zu 00001001_2 , 1100000011_2 , 10_2 , 0011001_{20} , 0100110110_2 , 01_2

Wir wandeln die Binärzahlen in Dezimalzahlen um: 18, 771, 50, 50, 310, 25

Aus den erhaltenen Zahlen ergeben sich die Koordinaten 18.77150, 50.31025

Funktion zur Erzeugung und Berechnung von Koordinaten aus der Internationalen Postleitzahl

Im Folgenden zwei Funktionen in C#. Die erste konstruiert die Internationale Postleitzahl, die zweite berechnet aus der Internationalen Postleitzahl die Koordinaten.

```
public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Coder = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F", "G", "H", "J",
        "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
            }
        }
    }
}
```

```

        TempHiLat = TempHiLat | OptionsLat;
    }

    TempLoLat = TempLoLat << 4;
}

ulong OptionsLon = 5;
Precision = Convert.ToInt32(StrsLon[1].Substring(3));
if (Precision < 125)
{
    OptionsLon = 0;
}
else if (Precision < 375)
{
    OptionsLon = 1;
}
else if (Precision < 625)
{
    OptionsLon = 2;
}
else if (Precision < 875)
{
    OptionsLon = 3;
}

ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
TempHiLon = TempHiLon << 32;
ulong TempLoLo = 0;
if (StrsLon.Length > 1)
{
    TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
    if (OptionsLon == 5)
    {
        TempLoLo++;
    }
    else
    {
        TempHiLon = TempHiLon | (OptionsLon * 4);
    }
}

TempLoLo = TempLoLo << 14;
}

ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

ulong Pos = 0;
string Res = "";
for (int i = 7; i >= 0; i--)
{
    Pos = UlongResHi / (ulong)Math.Pow(34, i);
    UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
    Res += Coder[Pos];
}

return Res;
}
catch
{
    return "";
}
}

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {

```

```

Postcode = Postcode.ToUpper();
string Coder = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
ulong Hi = 0;

for (int i = 0; i < 8; i++)
{
    Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7 - i));
}

ulong OptionLat = (Hi & 3);
double HalfLat = 0;
switch (OptionLat)
{
    case 1:
        HalfLat = 0.00025;
        break;
    case 2:
        HalfLat = 0.0005;
        break;
    case 3:
        HalfLat = 0.00075;
        break;
}

ulong OptionLon = (Hi & 12) >> 2;
double HalfLon = 0;
switch (OptionLon)
{
    case 1:
        HalfLon = 0.00025;
        break;
    case 2:
        HalfLon = 0.0005;
        break;
    case 3:
        HalfLon = 0.00075;
        break;
}

long LoLat = ((long)Hi >> 4) & 1023;
long HiLat = ((long)Hi >> 24) & 255;
if (HiLat > 180)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLat > 90)
{
    HiLat = HiLat - 180;
    Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
}
else
{
    Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
}

long LoLon = ((long)Hi >> 14) & 1023;
long HiLon = ((long)Hi >> 32);
if (HiLon > 360)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLon > 180)

```

```
{
    HiLon = HiLon - 360;
    Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
}
else
{
    Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
}

return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}
```